

ABSTRACT

Real-time object detection has become an important requirement in modern surveillance systems used in applications such as public safety, industrial monitoring, traffic management, and smart city infrastructure. Conventional surveillance systems based on CPUs and GPUs often face challenges related to high power consumption, increased processing latency, and limited efficiency when handling continuous high-resolution video streams. These limitations create a need for efficient hardware platforms capable of performing intelligent video processing with low latency and reliable real-time performance. Field Programmable Gate Arrays (FPGAs) have emerged as a promising solution due to their parallel processing capability, reconfigurability, deterministic execution, and energy-efficient operation.

This work presents an AI-enhanced FPGA-based smart surveillance system for real-time object detection using a lightweight YOLO-based convolutional neural network (CNN). The proposed system integrates a USB camera, FPGA processing platform, memory modules, display interface, and alert system to create a complete intelligent surveillance architecture. The camera continuously captures live video frames, which are transferred to the FPGA for further processing. Pre-processing operations such as resizing, normalization, and frame conversion are performed before feeding the data into the CNN-based object detection module. The FPGA utilizes hardware acceleration techniques including pipelining and parallel execution to improve inference speed and reduce computational delay.

The object detection engine identifies objects such as persons, chairs, and bottles by generating bounding boxes, confidence scores, and class labels in real time. The processed output is displayed through an HDMI monitor, while an alert mechanism using a buzzer and LED is activated whenever predefined objects or abnormal activities are detected. Communication interfaces such as Ethernet or UART can also be used for transmitting detection data to external systems for remote monitoring and storage.

Experimental results demonstrate that the proposed system achieves real-time performance at approximately 15 frames per second while maintaining stable detection accuracy under different test conditions. The FPGA-based implementation offers advantages such as low latency, reduced power consumption, and efficient resource utilization compared to conventional processing platforms. Overall, the proposed system demonstrates that combining FPGA acceleration with deep learning-based object detection provides an effective and scalable solution for intelligent real-time surveillance applications.

Chapter 1

INTRODUCTION

1.1 Overview

The rapid growth of intelligent surveillance technologies has significantly transformed the field of modern security and monitoring systems. Surveillance systems are widely used in public places, transportation networks, industries, commercial buildings, smart cities, military zones, and border security applications to ensure safety and monitor activities continuously. With the increasing deployment of high-resolution cameras and Internet-connected monitoring devices, a massive amount of video data is generated every second. Processing and analyzing this continuous stream of visual data in real time has become one of the major challenges in modern surveillance systems. Traditional surveillance systems mainly depend on manual monitoring, where human operators continuously observe video streams from multiple cameras. Such approaches are inefficient, time-consuming, and highly prone to human error, especially in large-scale monitoring environments. Therefore, there is a growing demand for intelligent surveillance systems capable of automatically detecting and analyzing objects in real time without constant human intervention.

Recent advancements in Artificial Intelligence (AI), Machine Learning (ML), and Computer Vision have greatly improved the capability of surveillance systems. In particular, Deep Learning techniques based on Convolutional Neural Networks (CNNs) have demonstrated remarkable performance in image classification, object recognition, and object detection tasks. CNN-based algorithms are capable of automatically learning features from image data, eliminating the need for manual feature extraction methods used in conventional image processing systems. As a result, deep learning-based object detection systems have become highly accurate and reliable for identifying objects such as humans, vehicles, animals, and suspicious items in surveillance applications.

Several advanced object detection algorithms have been developed in recent years, including Faster R-CNN, SSD (Single Shot Detector), and YOLO (You Only Look Once). Among these methods, the YOLO family of algorithms has gained significant popularity because of its ability to perform object detection in real time using a single-stage architecture. Unlike traditional two-stage detection methods, YOLO processes the entire image in a single pass, enabling faster inference and lower computational delay. Newer versions such as YOLOv3 and YOLOv4 have further improved detection accuracy, multi-scale feature extraction, and detection speed, making them highly suitable for real-time surveillance applications. Due to these advantages, YOLO-based models are widely adopted in intelligent monitoring systems, autonomous vehicles, industrial automation, and edge AI applications.

Although deep learning-based object detection systems provide high detection accuracy, deploying such models on conventional computing platforms presents several practical challenges. Central Processing Units (CPUs) generally lack sufficient parallel processing

capability to execute complex CNN operations efficiently in real time. Graphics Processing Units (GPUs), while offering better performance, consume high power and generate significant heat, making them unsuitable for low-power embedded systems and edge devices. In surveillance applications where continuous operation is required, power efficiency, low latency, and deterministic performance are critical requirements. Therefore, there is an increasing need for hardware platforms that can efficiently accelerate deep learning inference while maintaining low power consumption and stable real-time performance.

Field Programmable Gate Arrays (FPGAs) have emerged as an effective solution for accelerating AI-based applications due to their parallel processing capability, hardware reconfigurability, and energy-efficient operation. Unlike CPUs and GPUs, FPGAs allow developers to design custom hardware architectures optimized for specific applications. This enables efficient implementation of convolution operations, matrix multiplications, and data pipelines required for CNN-based object detection. FPGA platforms also support pipelining and parallel execution techniques, which significantly improve throughput and reduce processing latency. Additionally, FPGA-based systems provide deterministic execution, which is highly important in time-critical surveillance and security applications.

The integration of deep learning algorithms with FPGA hardware acceleration has opened new possibilities for real-time intelligent surveillance systems. By implementing CNN models directly on FPGA programmable logic, computationally intensive tasks can be executed efficiently with lower power consumption compared to traditional platforms. Optimization techniques such as quantization, pruning, and model compression further reduce memory usage and computational complexity, enabling deployment of lightweight neural networks on resource-constrained FPGA devices. Modern FPGA development frameworks and AI acceleration tools also simplify the deployment of deep learning models on embedded platforms, making FPGA-based AI systems more accessible for real-world applications.

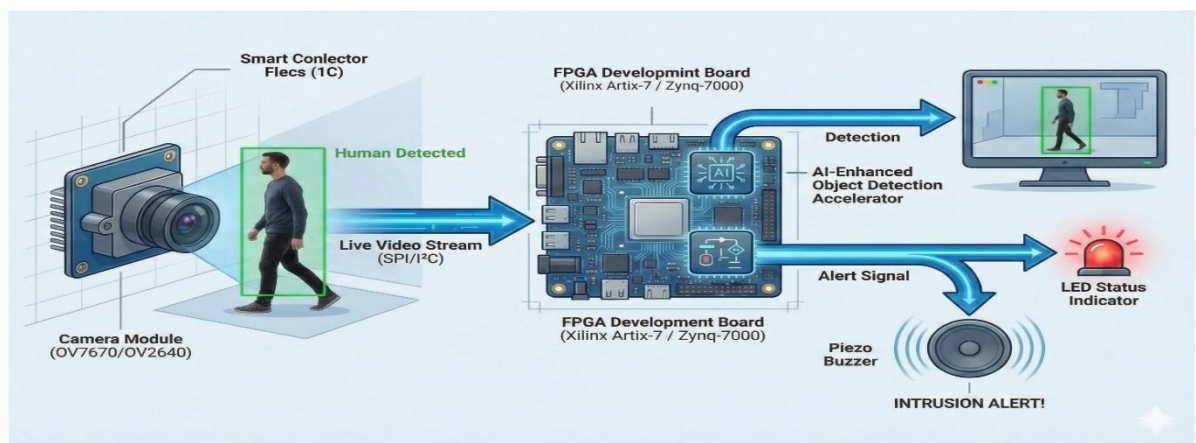


Fig.1 AI-Enhanced FPGA-Based Smart Surveillance System for Real-Time Object Detection

In this work, an AI-enhanced FPGA-based smart surveillance system for real-time object detection is proposed using a lightweight YOLO-based CNN model. The system is implemented using a PYNQ-Z2 FPGA platform integrated with multiple peripheral components including a USB camera, HDMI display interface, memory modules, Ethernet communication interface, and an alert system consisting of a buzzer and LED. The camera continuously captures live video frames from the surrounding environment, which are then transferred to the FPGA processing system for further analysis. The captured frames undergo pre-processing operations such as resizing, normalization, and image enhancement before being passed to the CNN-based object detection accelerator.

The object detection module implemented on the FPGA identifies objects in real time and generates bounding boxes, class labels, and confidence scores for detected objects. The detection results are displayed on a monitor through an HDMI interface, enabling continuous visual monitoring. Additionally, the system includes a decision and alert module that activates a buzzer whenever predefined objects or suspicious activities are detected. Communication interfaces such as UART or Ethernet can also be used to transfer detection results to external systems, cloud platforms, or monitoring servers for remote access and data storage.

The main objective of the proposed work is to develop an efficient, scalable, and low-power intelligent surveillance system capable of performing reliable real-time object detection using FPGA hardware acceleration. By combining deep learning-based object detection with embedded FPGA technology, the proposed system aims to achieve low latency, improved throughput, efficient resource utilization, and reduced power consumption suitable for edge-based surveillance applications. Furthermore, the architecture can be extended in the future to support advanced features such as multi-object tracking, behavior analysis, cloud integration, and smart city surveillance systems.

1.2 Limitations of Existing Systems

Despite the rapid advancements in deep learning-based surveillance technologies, existing object detection systems still face several limitations when deployed in real-time embedded environments. Traditional CPU-based systems often suffer from slow processing speed and high latency due to limited parallel computation capability. Although GPU-based implementations provide improved performance, they consume significant power and require expensive hardware resources, making them unsuitable for low-power edge applications and continuous surveillance systems.



Fig.2 Limitations of Traditional CPU/GPU-Based Surveillance Systems in Real-Time Applications

Many conventional surveillance systems also struggle to process multiple high-resolution video streams in real time, leading to frame drops and delayed detection responses. In addition, deep learning models with large network architectures require high memory capacity and computational resources, which increases hardware complexity and overall system cost. These challenges become more critical in embedded systems where resource availability is limited.

Another major limitation is the lack of energy-efficient hardware acceleration in traditional monitoring platforms. Continuous operation of GPU-based systems generates excessive heat and increases operational costs, especially in large-scale surveillance deployments. Furthermore, some existing systems experience reduced detection accuracy under varying lighting conditions, object occlusions, and dynamic backgrounds.

Many surveillance systems are also designed only for software-based execution without efficient hardware–software integration. This reduces throughput and limits the capability of achieving deterministic real-time performance. Additionally, conventional systems often lack integrated alert mechanisms and efficient edge-level processing, resulting in dependency on cloud servers and network connectivity.

1.3 Proposed System

The proposed system presents an AI-enhanced FPGA-based smart surveillance architecture designed for real-time object detection and monitoring. The system integrates deep learning techniques with FPGA hardware acceleration to achieve efficient processing, reduced latency, and low power consumption suitable for embedded and edge computing applications. The proposed architecture mainly consists of a camera module, FPGA development board, CNN-based object detection accelerator, memory interface, display unit, communication interface, and alert system.

The system uses a USB camera to continuously capture live video frames from the monitoring environment. The captured video stream is transferred to the FPGA platform through a high-speed communication interface. In this work, a PYNQ-Z2 FPGA board is used as the primary processing platform due to its hardware reconfigurability and parallel processing capability. The FPGA integrates both the Processing System (PS) and Programmable Logic (PL), enabling efficient hardware–software co-design.

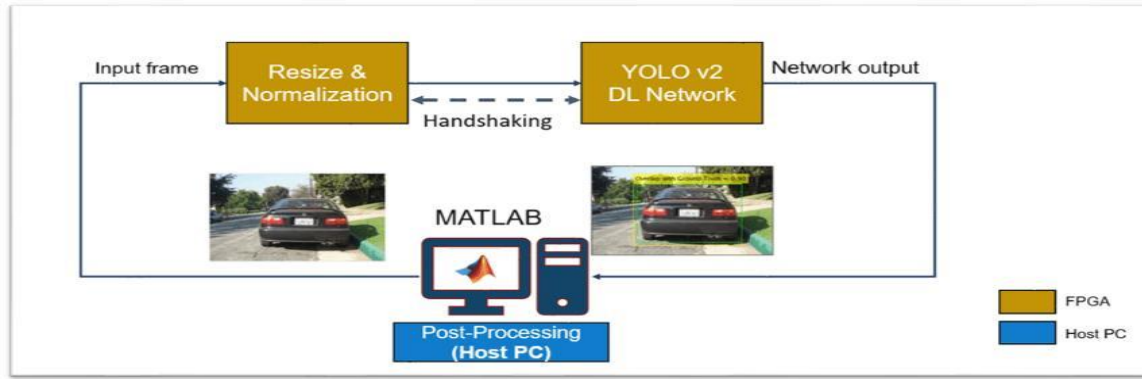


Fig. 3. Proposed FPGA-Based Smart Surveillance System Architecture

Initially, the incoming video frames undergo image pre-processing operations such as resizing, normalization, and frame conversion to prepare the data for object detection. These operations are implemented in FPGA programmable logic to reduce the processing load on the CPU. The pre-processed frames are then passed to a lightweight YOLO-based CNN object detection module implemented on the FPGA accelerator.

The CNN accelerator performs real-time inference by identifying objects within each frame and generating bounding boxes, class labels, and confidence scores. FPGA-based acceleration techniques such as pipelining and parallel execution are utilized to improve throughput and minimize inference latency. Intermediate data and feature maps are stored using BRAM and DDR memory interfaces for efficient memory access.

The detected output is forwarded to the decision and alert module, where predefined rules are applied to identify suspicious or important objects. Whenever a specific object is detected, the system activates a buzzer and LED alert mechanism through GPIO interfaces. Simultaneously, the processed output is displayed on a monitoring screen through an HDMI interface, enabling real-time visualization of detection results.

In addition, communication interfaces such as Ethernet or UART can be integrated for transmitting detection data to external servers or cloud platforms for remote monitoring and storage. The entire system operates continuously in a streaming pipeline manner, ensuring reliable real-time surveillance performance.

The proposed FPGA-based surveillance system provides several advantages compared to traditional CPU- and GPU-based implementations, including lower power consumption, deterministic execution, reduced latency, and efficient hardware resource utilization. These characteristics make the system suitable for smart surveillance, industrial monitoring, traffic analysis, and edge AI applications.

1.4 Role of FPGA in the Proposed System

The FPGA plays a major role in the proposed smart surveillance system by providing hardware acceleration for real-time object detection and video processing. Unlike conventional CPU-based systems, the FPGA offers parallel processing capability, enabling multiple operations to

be executed simultaneously. This significantly improves processing speed and reduces system latency, which is essential for real-time surveillance applications.

In the proposed system, the FPGA acts as the central processing platform responsible for handling image pre-processing, CNN-based object detection, memory management, and data transfer operations. The programmable logic (PL) section of the FPGA accelerates computationally intensive tasks such as convolution, feature extraction, and inference operations used in the YOLO-based object detection model. By implementing these operations directly in hardware, the system achieves faster execution compared to software-only approaches.

The FPGA also supports pipelining techniques, where multiple stages of processing are executed concurrently. This improves throughput and enables continuous processing of incoming video frames without significant delay. Additionally, the FPGA provides deterministic execution, ensuring stable and predictable performance for time-critical surveillance systems.

Another important role of the FPGA is efficient hardware resource utilization and low power operation. Compared to GPU-based systems, FPGA implementations consume lower power while maintaining reliable real-time performance, making them suitable for embedded and edge AI applications. The FPGA also interfaces with peripheral components such as the USB camera, HDMI display, memory modules, Ethernet communication interface, and buzzer-based alert system.

1.5 System Safety and Misuse Prevention

The proposed FPGA-based smart surveillance system is designed with safety and reliability considerations to ensure secure and responsible operation in real-time monitoring environments. Since the system performs continuous object detection and surveillance, appropriate measures are necessary to prevent system failure, false alerts, unauthorized access, and misuse of monitored data.

One of the important safety aspects of the system is reliable real-time operation. The FPGA platform provides deterministic processing and stable execution, reducing the possibility of system crashes or unpredictable delays during continuous monitoring. The use of hardware acceleration also minimizes processing bottlenecks, ensuring uninterrupted surveillance performance even under high workloads.

To prevent false detections and unnecessary alarms, the system uses confidence-based object classification. Alerts are generated only when the detected object confidence exceeds a predefined threshold value. This reduces the occurrence of false positives and improves system reliability in practical applications. In addition, the decision logic module verifies detection results before activating the buzzer or alert mechanism.

The system also includes hardware-level protection for connected peripherals such as the camera, HDMI display, and buzzer interface. Proper voltage regulation and controlled GPIO interfacing are maintained to prevent electrical damage to external devices. Memory

management techniques are also used to avoid buffer overflow and frame synchronization issues during continuous video processing.

From a security perspective, the communication interface can be configured with authentication and restricted network access to prevent unauthorized users from accessing surveillance data. Data transmitted through Ethernet or UART interfaces can also be encrypted for secure communication in remote monitoring applications.

To avoid misuse of the surveillance system, access to detection results and monitoring interfaces should be limited to authorized personnel only. The system is intended strictly for security, monitoring, and safety-related applications and should not be used for unauthorized tracking or violation of privacy. Proper deployment policies and ethical guidelines must be followed while using AI-based surveillance systems in public or private environments.

Overall, the proposed system incorporates both hardware and software safety mechanisms to ensure reliable operation, prevent misuse, and maintain secure real-time surveillance performance.

1.6 Objectives of the Project

The main objective of this project is to design and implement an AI-enhanced FPGA-based smart surveillance system capable of performing real-time object detection with improved processing efficiency and reduced power consumption. The system aims to combine deep learning-based object detection techniques with FPGA hardware acceleration to achieve reliable and low-latency surveillance performance suitable for embedded and edge AI applications.

The specific objectives of the project are as follows:

- To develop a real-time intelligent surveillance system using a YOLO-based convolutional neural network (CNN) model.
- To implement the object detection framework on a PYNQ-Z2 FPGA platform for efficient hardware acceleration.
- To capture and process live video streams continuously using a USB camera interface.
- To perform image pre-processing operations such as resizing, normalization, and frame conversion before CNN inference.
- To utilize FPGA parallel processing and pipelining techniques for improving inference speed and reducing latency.
- To detect and classify multiple objects such as persons, chairs, and bottles in real time with acceptable accuracy.
- To generate bounding boxes, class labels, and confidence scores for detected objects during live monitoring.
- To display real-time detection output on a monitor through an HDMI interface.
- To integrate an alert mechanism using a buzzer and LED for notifying suspicious activities or predefined object detection.
- To optimize memory usage and hardware resource utilization using FPGA-based design techniques.

- To reduce overall power consumption compared to traditional CPU- and GPU-based object detection systems.
- To enable communication with external systems using Ethernet or UART interfaces for remote monitoring and data transfer.
- To design a scalable surveillance architecture that can be extended for future smart city, industrial monitoring, and edge AI applications.

1.7 Challenges in Emergency Situations

Real-time surveillance systems face several challenges when operating in emergency situations such as accidents, unauthorized intrusions, industrial hazards, and public security threats. During such conditions, the system must process video data quickly and generate immediate alerts to support rapid decision-making. Traditional CPU-based systems often suffer from high processing latency and limited parallel processing capability, which can delay object detection and reduce system efficiency during critical situations.

Environmental conditions also affect the performance of surveillance systems. Factors such as poor lighting, shadows, smoke, rain, motion blur, and object occlusion can reduce detection accuracy and lead to false alarms or missed detections. In crowded environments, detecting and tracking multiple objects simultaneously becomes more difficult and computationally intensive.

Another major challenge is maintaining continuous operation with low power consumption. GPU-based systems provide high computational performance but consume significant power and generate excessive heat, making them unsuitable for long-term edge surveillance applications. In addition, cloud-dependent surveillance systems may experience communication delays or network failures during emergencies, affecting real-time response capability.

To overcome these challenges, the proposed FPGA-based surveillance system utilizes hardware acceleration, parallel processing, and pipelining techniques to achieve low-latency and energy-efficient real-time object detection. These features improve system reliability and make the architecture suitable for emergency and safety-critical applications.

1.8 Need for Intelligent and Automated Systems

The increasing demand for advanced security and monitoring has created the need for intelligent and automated surveillance systems capable of operating efficiently in real time. Traditional surveillance systems mainly rely on continuous human monitoring, which becomes difficult and inefficient when handling multiple video streams simultaneously. Human operators may miss important events due to fatigue, delayed response, or limited attention, especially in large-scale surveillance environments such as airports, industries, smart cities, and public transportation systems. With the rapid growth of Artificial Intelligence (AI) and Computer Vision technologies, automated surveillance systems can now detect, classify, and analyze objects without constant human intervention. Intelligent systems improve monitoring

accuracy, reduce response time, and enable real-time decision-making during critical situations. Deep learning-based object detection models such as YOLO can identify suspicious activities, unauthorized access, and important objects with high efficiency.

In addition, automated systems reduce operational cost and improve reliability compared to traditional monitoring approaches. Integrating AI with FPGA-based hardware acceleration further enhances system performance by providing low-latency processing, efficient resource utilization, and reduced power consumption. Therefore, intelligent and automated surveillance systems have become essential for modern security, industrial monitoring, and edge AI applications.

1.9 Real-World Applications and Impact

The proposed FPGA-based smart surveillance system has a wide range of real-world applications in security, monitoring, and intelligent automation. In public safety systems, the technology can be used in airports, railway stations, shopping malls, and smart cities for real-time monitoring and detection of suspicious activities. Industrial environments can utilize the system for worker safety monitoring, restricted-area surveillance, and hazard detection. Traffic monitoring systems can also use real-time object detection for vehicle analysis, congestion monitoring, and accident prevention.

The system is highly suitable for edge AI applications due to its low power consumption and real-time processing capability. FPGA-based acceleration enables efficient deployment in embedded systems where continuous operation and low latency are critical. The integrated alert mechanism improves response time by generating immediate notifications during abnormal situations.

The impact of the proposed system includes improved surveillance efficiency, reduced dependency on human monitoring, faster emergency response, and enhanced security management. By combining deep learning with FPGA hardware acceleration, the system provides a scalable and energy-efficient solution for modern intelligent surveillance applications.

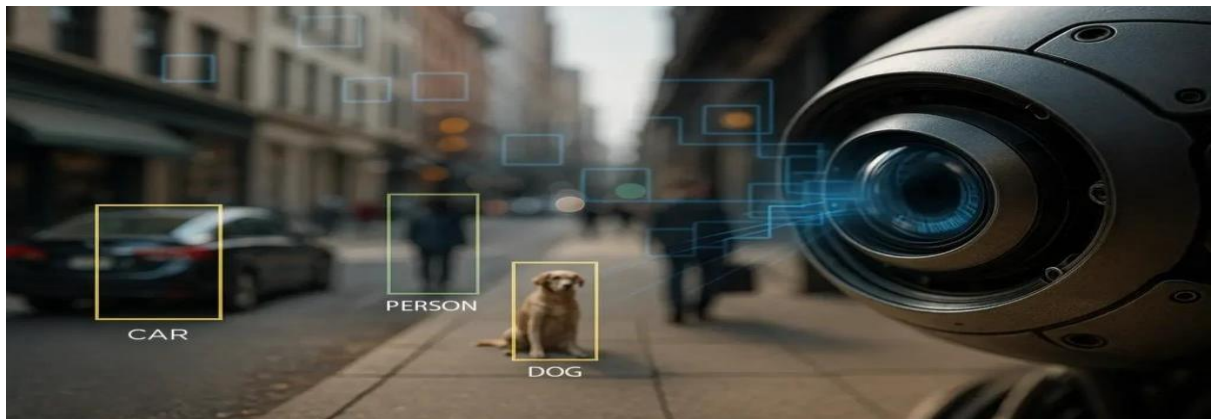


Fig. 4. Real-World Applications of AI-Enhanced FPGA-Based Smart Surveillance Systems

Chapter 2

LITERATURE SURVEY

Recent advancements in Artificial Intelligence (AI), Deep Learning, and Computer Vision have significantly improved the performance of real-time object detection systems used in surveillance and monitoring applications. Researchers have focused on developing efficient object detection algorithms capable of identifying and classifying objects with high accuracy and reduced processing delay. Deep learning models such as Convolutional Neural Networks (CNNs) and YOLO-based architectures have become widely popular due to their ability to perform fast and reliable detection in real-time environments.

However, implementing these computationally intensive algorithms on conventional CPU- and GPU-based systems introduces challenges such as high power consumption, increased latency, and hardware complexity. These limitations become more critical in embedded and edge AI applications where continuous operation, low power usage, and deterministic performance are essential.

To address these challenges, several research works have explored FPGA-based acceleration techniques, hardware–software co-design methods, lightweight CNN architectures, and quantization approaches for efficient deep learning inference. FPGA platforms provide advantages such as parallel processing capability, hardware reconfigurability, low-latency execution, and optimized resource utilization, making them suitable for intelligent surveillance systems.

The following section discusses important research contributions related to FPGA-based object detection, CNN acceleration, YOLO optimization, and embedded AI systems for real-time surveillance applications.

[1] Anupreetham, R. R. Mudiya, and K. Kavi

This work presents a high-throughput FPGA-based object detection system using an algorithm–hardware co-design approach. The authors optimized both the neural network architecture and FPGA hardware implementation to improve processing efficiency and throughput. Their design demonstrates that FPGA platforms can achieve low-latency performance with better energy efficiency compared to conventional CPU-based systems. The study also highlights the importance of balancing hardware resource utilization and detection accuracy. The proposed approach is suitable for real-time embedded surveillance applications.

[2] J. Redmon et al. – YOLO

The YOLO (You Only Look Once) algorithm introduced a single-stage object detection framework capable of performing real-time detection with high speed. Unlike traditional two-stage detectors, YOLO processes the entire image in a single pass, reducing computational complexity and inference delay. The model divides the image into grids and predicts bounding boxes and class probabilities simultaneously. This approach significantly improves detection speed while maintaining acceptable accuracy. YOLO became one of the most widely used object detection algorithms for real-time applications.

[3] J. Redmon and A. Farhadi – YOLOv3

YOLOv3 improved the original YOLO architecture by introducing multi-scale feature extraction and residual network structures. The model enhances detection performance, especially for small objects, while maintaining real-time inference capability. Darknet-53 was used as the backbone network to improve feature representation and accuracy. The study demonstrated better precision and improved object localization compared to previous versions. YOLOv3 is widely used in surveillance and edge AI applications due to its balance between speed and performance.

[4] A. Bochkovskiy et al. – YOLOv4

YOLOv4 introduced several optimization techniques to improve both object detection accuracy and inference speed. The model combines advanced training strategies, feature aggregation methods, and improved backbone networks for enhanced performance. The authors focused on achieving high detection efficiency while maintaining compatibility with embedded and real-time systems. YOLOv4 demonstrated significant improvements over earlier YOLO versions in terms of precision and robustness. It is considered suitable for real-time surveillance and intelligent monitoring applications.

[5] M. Liu et al.

This research proposed an efficient real-time object detection framework for resource-constrained hardware devices using hardware–software co-design techniques. The study emphasized optimizing memory access and computational efficiency for embedded AI systems. The authors demonstrated that efficient task partitioning between hardware and software can improve real-time performance significantly. Their work also discussed the trade-off between hardware complexity and detection accuracy. The framework is suitable for FPGA-based edge AI applications.

[6] S. M. Sali et al.

The authors developed an FPGA-based CNN system capable of real-time object detection, classification, and tracking. The proposed architecture utilizes FPGA parallelism to accelerate deep learning inference while reducing power consumption. The study demonstrated stable low-latency performance suitable for autonomous and surveillance applications. The implementation also showed improved throughput compared to traditional software-based systems. This work highlights the effectiveness of FPGA acceleration for real-time AI processing.

[7] Z. Zhang et al. – FitNN

FitNN is a lightweight FPGA-based CNN accelerator designed specifically for edge computing devices with limited hardware resources. The proposed architecture focuses on reducing FPGA resource utilization while maintaining acceptable inference performance. The study demonstrated that optimized FPGA accelerators can efficiently support CNN-based applications in embedded systems. The framework also improves power efficiency and scalability for real-time object detection tasks. FitNN is suitable for low-power intelligent surveillance applications.

[8] C. Ding et al. – REQ-YOLO

REQ-YOLO introduced a resource-aware quantization framework for deploying YOLO-based object detection models on FPGA platforms. The authors used low-precision computations to reduce memory usage and computational complexity. Quantization techniques significantly improved hardware efficiency while maintaining reasonable detection accuracy. The proposed framework demonstrated reduced power consumption and faster inference performance on embedded FPGA systems. This work is important for real-time FPGA-based deep learning applications.

[9] T. B. Preuber et al.

This research focused on implementing quantized neural networks on FPGA devices for efficient deep learning inference. The authors demonstrated that low-precision arithmetic operations can significantly reduce computational complexity, memory usage, and power consumption without major loss in detection accuracy. The study highlighted the advantages of FPGA platforms in supporting parallel execution and hardware optimization for neural network processing. The proposed implementation achieved improved throughput and energy efficiency compared to traditional software-based approaches. This work is important for developing lightweight FPGA-based AI systems for edge applications.

[10] R. Al Amin et al.

The authors proposed an FPGA-based real-time object detection and classification system using the YOLO algorithm for edge computing applications. Their work focused on achieving real-time performance with reduced latency and lower power consumption compared to GPU-based implementations. The FPGA architecture efficiently accelerated CNN inference operations and supported continuous video stream processing. Experimental results demonstrated stable detection performance suitable for intelligent surveillance systems. The study confirmed that FPGA acceleration is highly effective for embedded real-time monitoring applications.

[11] S. H. Hozhabr and R. Giorgi

This survey paper reviewed various FPGA-based real-time object detection systems and discussed the advantages and challenges of hardware acceleration for deep learning

applications. The authors analyzed different CNN acceleration techniques, FPGA architectures, optimization methods, and embedded AI frameworks. The study emphasized the importance of balancing detection accuracy, hardware resource utilization, power efficiency, and processing speed. It also highlighted the growing role of FPGA platforms in edge computing and intelligent surveillance applications. The paper provides valuable insights into future research directions in FPGA-based object detection systems.

[12] Xilinx Deep Learning Processor Unit (DPU)

The Xilinx DPU is a dedicated FPGA-based hardware accelerator designed for deep learning inference applications. It simplifies the deployment of CNN models on FPGA platforms by providing optimized processing architectures for neural network execution. The framework supports various AI models and enables efficient acceleration of convolution operations with reduced latency. The DPU architecture improves throughput and power efficiency compared to software-only implementations. This platform is widely used in embedded AI and real-time edge computing systems.

[13] M. Blott et al. – FINN-R

FINN-R is an end-to-end deep learning framework designed for implementing quantized neural networks on FPGA devices. The framework automates the conversion of neural network models into FPGA-compatible hardware architectures, reducing development complexity. FINN-R supports low-precision inference techniques, enabling efficient deployment of lightweight CNN models with reduced memory usage and power consumption. The study demonstrated that FPGA-based quantized inference can achieve high throughput and low latency for real-time applications. This framework is highly useful for edge AI and embedded surveillance systems.

Author & Year	Methodology / Model	Key Findings & Strengths	Limitations
Anupreetham et al. (2024)	FPGA-based object detection using hardware–software co-design	Achieved high throughput and improved energy efficiency for real-time applications	FPGA resource utilization increases for complex CNN models
Redmon et al. (2016)	YOLO single-stage object detection model	Enabled fast real-time object detection with reduced inference delay	Lower accuracy for small object detection
Redmon & Farhadi (2018)	YOLOv3 with multi-scale feature extraction	Improved object localization and small object detection accuracy	Higher computational complexity compared to earlier YOLO versions

Bochkovskiy et al. (2020)	YOLOv4 optimized CNN architecture	Improved detection speed and accuracy using advanced training techniques	Requires optimized hardware for efficient deployment
Liu et al. (2024)	Hardware–software co-design framework for embedded AI	Efficient real-time object detection on resource-constrained devices	Trade-off between hardware complexity and detection accuracy
Sali et al. (2025)	FPGA-based CNN detection and tracking system	Achieved deterministic low-latency performance with high throughput	Limited flexibility for large-scale CNN models
Zhang et al. (2022)	FitNN lightweight FPGA CNN accelerator	Reduced FPGA resource utilization and improved power efficiency	Lower adaptability for different network architectures
Ding et al. (2019)	REQ-YOLO quantization framework	Reduced memory usage and computational complexity on FPGA	Slight reduction in detection accuracy due to quantization
Preußner et al. (2018)	Quantized neural network inference on FPGA	Improved throughput and power efficiency using low-precision arithmetic	Accuracy degradation in highly quantized models
Al Amin et al. (2024)	FPGA-based YOLO system for edge computing	Achieved real-time object detection with reduced power consumption	Performance depends on FPGA hardware resources
Hozhabr & Giorgi (2025)	Survey on FPGA-based object detection systems	Provided comprehensive analysis of FPGA acceleration techniques	Standardization challenges across different FPGA platforms
Xilinx Inc. (2023)	FPGA Deep Learning Processor Unit (DPU)	Simplified deployment of CNN models on FPGA hardware	Reduced customization compared to manual FPGA implementation
Blott et al. (2018)	FINN-R framework for quantized neural networks	Automated FPGA deployment of lightweight CNN models	Limited support for large complex neural networks
Han et al. (2016)	Deep compression techniques for CNNs	Reduced model size and memory requirements significantly	Compression may affect inference precision
Sze et al. (2017)	Efficient deep neural network processing techniques	Optimized memory access and computation efficiency	Hardware optimization complexity for large-scale systems

Table 1. Comparative Analysis of Existing FPGA-Based Object Detection and Surveillance Systems

Chapter 3

PROPOSED WORK

3.1 Introduction to the Proposed System

The proposed work presents an AI-enhanced FPGA-based smart surveillance system designed for real-time object detection and intelligent monitoring applications. The system combines deep learning techniques with FPGA hardware acceleration to achieve efficient processing, low latency, and reduced power consumption suitable for embedded and edge AI environments. The proposed architecture is developed to overcome the limitations of conventional CPU- and GPU-based surveillance systems, which often suffer from high computational complexity, increased power usage, and delayed response during continuous real-time monitoring.

The system utilizes a PYNQ-Z2 FPGA platform integrated with a USB camera, HDMI display, memory modules, communication interfaces, and an alert mechanism. The USB camera continuously captures live video frames from the surrounding environment and transfers them to the FPGA processing system. The captured frames undergo pre-processing operations such as resizing, normalization, and image enhancement before being passed to the CNN-based object detection accelerator.

A lightweight YOLO-based convolutional neural network model is implemented for real-time object detection. The FPGA programmable logic accelerates computationally intensive CNN operations using parallel processing and pipelining techniques, enabling faster inference and deterministic execution. The system is capable of detecting multiple objects such as persons, chairs, and bottles while generating bounding boxes, confidence scores, and class labels in real time.

The processed output is displayed on a monitor through an HDMI interface for continuous monitoring. In addition, the system includes a buzzer and LED-based alert mechanism that generates notifications whenever predefined objects or suspicious activities are detected. Communication interfaces such as Ethernet or UART can also be used for transmitting detection data to external systems or cloud servers.

The proposed system aims to provide an efficient, scalable, and low-power intelligent surveillance solution suitable for smart city infrastructure, industrial monitoring, public safety systems, and edge computing applications.

3.2 System Design Philosophy and Approach

The proposed smart surveillance system is implemented using the PYNQ-Z2 FPGA board, which integrates an ARM-based Processing System (PS) and Programmable Logic (PL) for efficient hardware–software co-design. The system is designed to perform real-time object detection using a lightweight YOLO-based CNN model accelerated on FPGA hardware.

The system begins with a USB camera that continuously captures live video frames from the monitoring environment. The captured video data is transferred to the PYNQ-Z2 FPGA board through the USB interface. Inside the FPGA, the incoming frames undergo image pre-processing operations such as resizing, normalization, and frame conversion to improve detection performance and reduce computational complexity.

After pre-processing, the frames are forwarded to the CNN-based object detection accelerator implemented in the programmable logic section of the FPGA. The YOLO model processes each frame in real time and generates bounding boxes, object labels, and confidence scores. FPGA parallel processing and pipelining techniques are utilized to improve throughput and reduce processing latency.

The detection results are analyzed by the decision logic module running on the processing system. When predefined objects or suspicious activities are detected, the system activates a buzzer and LED alert mechanism through GPIO interfaces. Simultaneously, the processed output is displayed on a monitor using the HDMI interface available on the PYNQ-Z2 board.

Additionally, communication interfaces such as Ethernet or UART can be used to transmit detection results to external systems or monitoring servers. The modular architecture of the system enables continuous real-time surveillance with efficient hardware resource utilization and reduced power consumption.

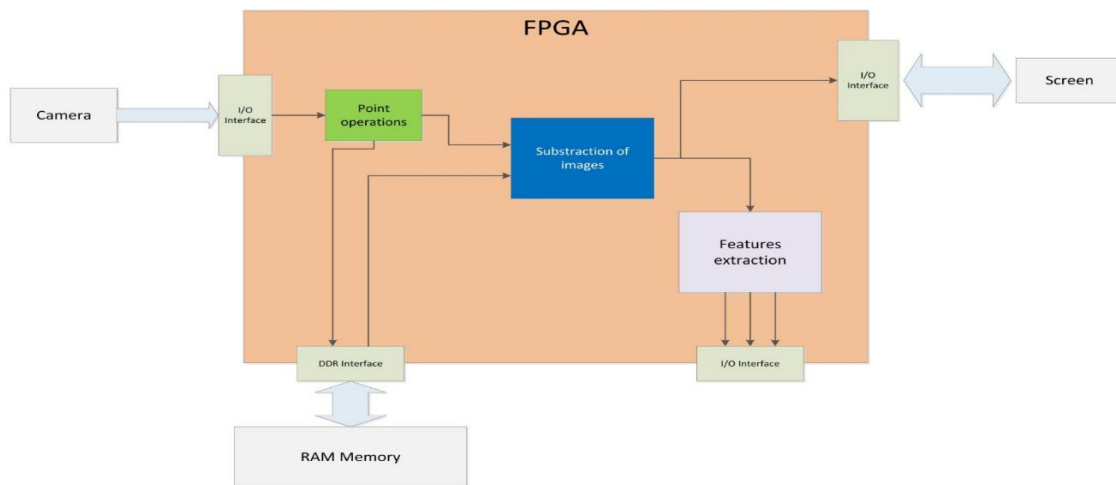


Fig.5. Block diagram of the proposed system

3.3 System Architecture Description

The proposed FPGA-based smart surveillance system is designed to perform efficient real-time object detection using a hardware–software co-design methodology. The architecture combines image acquisition, FPGA-based processing, CNN inference, memory management, display interfacing, and alert generation modules to achieve low-latency and energy-efficient surveillance performance. The complete system is implemented on the

PYNQ-Z2 platform, which integrates programmable logic and ARM processing capabilities for embedded AI applications.

The overall architecture of the proposed system is divided into the following functional modules:

3.3.1 Camera and Video Acquisition Module

The surveillance system utilizes a USB camera to continuously capture live video streams from the monitoring environment. The captured frames are transferred to the FPGA board through high-speed communication interfaces. This module ensures continuous frame acquisition and uninterrupted video streaming required for real-time monitoring applications. Efficient frame transfer minimizes latency and improves overall system performance.

3.3.2 Image Pre-processing Module

The acquired video frames undergo several image pre-processing operations such as resizing, normalization, frame conversion, and noise reduction. These operations improve image quality and prepare the input frames for CNN-based object detection. The pre-processing stage also reduces computational complexity and improves detection efficiency. FPGA parallelism is utilized to accelerate these operations for high-speed processing.

3.3.3 CNN-Based Object Detection Module

The object detection stage is implemented using a lightweight YOLO-based Convolutional Neural Network (CNN). This module performs feature extraction, object classification, and bounding box generation for detected objects in real time. The FPGA accelerates CNN inference using pipelining and parallel processing techniques, significantly reducing inference latency. The system is capable of detecting objects such as persons, chairs, and bottles with confidence scores and localization information.

3.3.4 Memory Management Module

DDR memory interfaces are used for temporary storage of video frames, intermediate feature maps, and processed output data. Efficient memory access and buffering mechanisms help maintain continuous real-time processing without frame loss. Optimized memory management also improves hardware resource utilization and reduces communication bottlenecks between processing modules.

3.3.5 Display and Monitoring Module

The processed output frames containing detected objects and bounding boxes are transmitted to a display monitor through the HDMI/DVI display controller interface. This module enables continuous real-time visualization of surveillance results. The monitoring display provides live detection output along with object labels and confidence values for user observation.

3.3.6 Alert and Decision Module

The decision module analyzes the detection results generated by the CNN accelerator. When suspicious activities or predefined object conditions are detected, the system activates a buzzer and LED-based alert mechanism through GPIO interfaces. This module

improves emergency response capability and enhances intelligent surveillance functionality.

3.3.7 Communication Interface Module

The architecture also supports communication interfaces such as Ethernet, UART, and Wi-Fi for transmitting detection data to external systems or remote monitoring servers. These interfaces enable remote access, data logging, and integration with IoT-based smart surveillance applications.

3.3.8 Overall System Operation

The complete system operates in a continuous streaming pipeline manner, where video frames are captured, processed, analyzed, and displayed in real time. FPGA-based parallel processing and hardware acceleration techniques improve throughput while reducing latency and power consumption. The modular design approach also simplifies system integration, maintenance, scalability, and future enhancements for advanced surveillance applications.

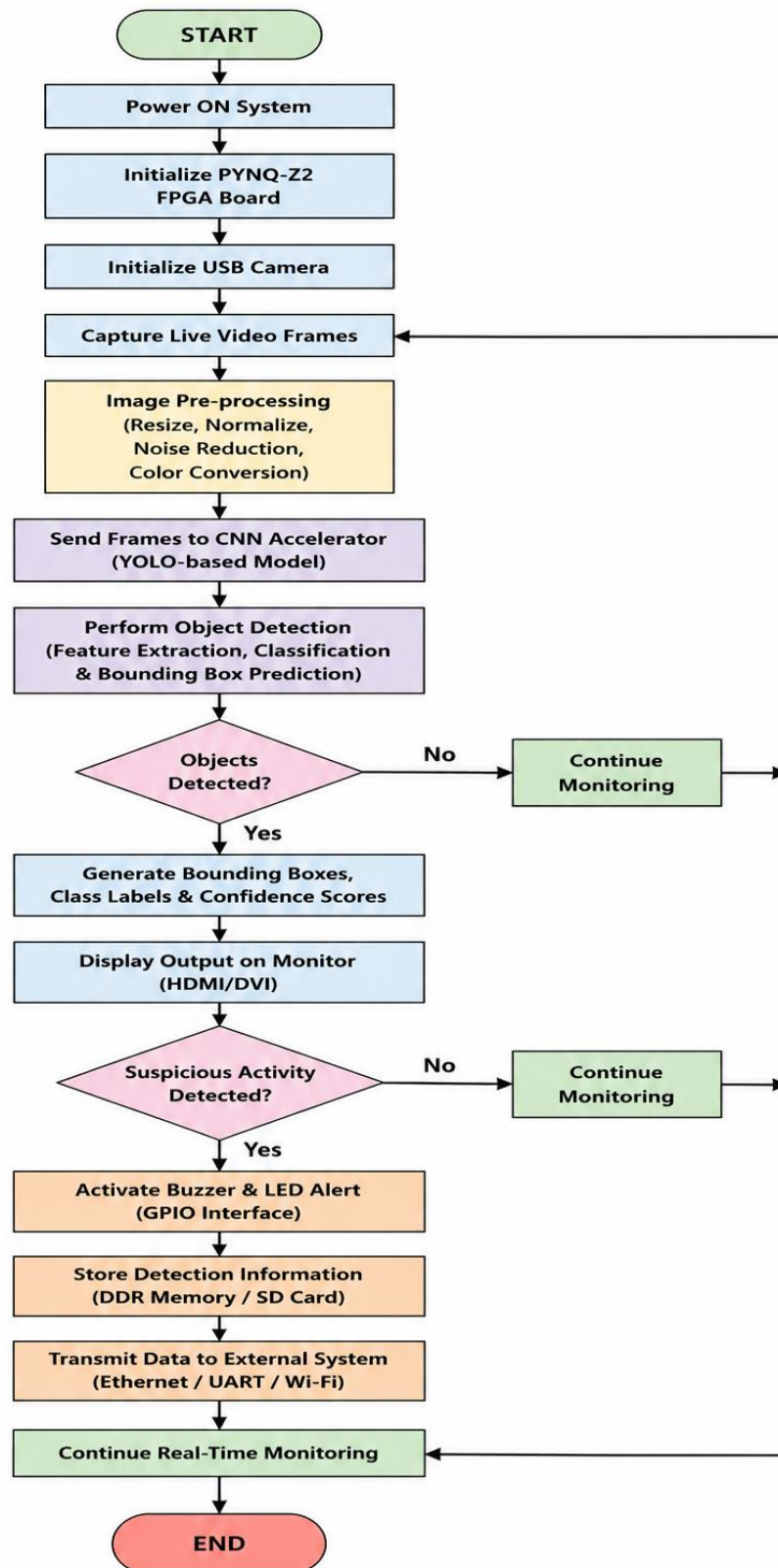


Fig.6. Flow chart of the proposed system

3.4 Geofencing-Based Detection Mechanism

The geofencing-based detection mechanism is used to define a virtual boundary around a specific monitoring area for intelligent surveillance and automated event detection. In the proposed system, geofencing enables the surveillance architecture to monitor predefined zones and generate alerts whenever an object or person enters, exits, or remains within the restricted area. This mechanism improves security monitoring and enhances automated decision-making capability in real-time environments.

The surveillance camera continuously captures live video frames and transfers them to the FPGA-based processing system. The YOLO-based CNN model detects and classifies objects present in the monitored region. The system then compares the detected object position with the predefined geofenced boundary coordinates stored in the FPGA memory. If an object crosses the restricted virtual boundary or enters a prohibited zone, the decision module immediately triggers an alert signal.

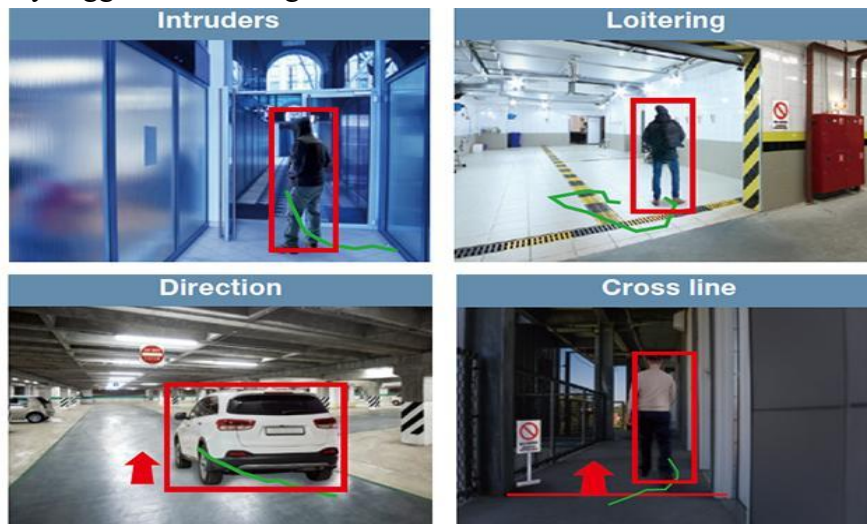


Fig. 7. Geofencing-Based Intrusion and Restricted Area Detection Mechanism

The alert mechanism activates the buzzer and LED notification system while simultaneously transmitting warning information to external monitoring systems through Ethernet, UART, or Wi-Fi communication interfaces. The geofencing logic can also be configured to monitor object movement direction, duration inside the zone, and repeated intrusions for advanced surveillance applications.

The FPGA-based implementation enables low-latency geofencing analysis through parallel processing and hardware acceleration. This improves response time and supports continuous real-time monitoring without significant processing delay. The proposed geofencing mechanism is highly useful in smart city surveillance, industrial safety monitoring, restricted-area protection, border security, and intelligent traffic management systems.

3.5 Data Processing and Decision-Making Logic

The proposed surveillance system processes live video frames captured from the USB camera using FPGA-based parallel processing techniques. Initially, the frames undergo image pre-processing operations such as resizing, normalization, and noise reduction to improve detection accuracy. The processed frames are then forwarded to the YOLO-based CNN accelerator for feature extraction, object classification, and bounding box generation. Detection results including object labels and confidence scores are analyzed by the decision-making module. If suspicious objects or restricted-area violations are detected, the system activates buzzer and LED alerts while transmitting notification data to external monitoring systems for real-time surveillance and emergency response.

3.6 Multi-Object Detection Strategy

The proposed surveillance system is capable of detecting multiple objects simultaneously while capturing live video frames. The YOLO-based CNN model processes the entire image frame in a single pass and identifies multiple objects present in the scene. For each detected object, the system generates bounding boxes, class labels, and confidence scores independently. The FPGA-based parallel processing architecture enables efficient handling of multiple detections without significant processing delay. When several objects are detected at the same time, the decision module analyzes all detection results and prioritizes alert generation based on predefined conditions. This strategy improves real-time monitoring capability and ensures reliable surveillance performance in crowded or dynamic environments.

3.7 System Reset and Recovery Mechanism

The proposed FPGA-based smart surveillance system includes a reset and recovery mechanism to ensure reliable continuous operation during unexpected failures or processing interruptions. The system continuously monitors the status of video acquisition, object detection, memory access, and communication modules during real-time execution. If errors such as frame loss, communication failure, or processing delay are detected, the system automatically reinitializes the affected module without interrupting the complete surveillance operation.

A hardware reset mechanism is implemented within the FPGA to restore normal functionality during abnormal conditions. The watchdog and monitoring logic help detect system instability and trigger recovery procedures when required. After recovery, the system resumes video capture, object detection, and alert generation operations automatically. This mechanism improves system reliability, reduces downtime, and ensures stable long-duration surveillance performance in real-time monitoring environments.

3.8 Hardware Implementation

The hardware implementation of the proposed smart surveillance system is carried out using the PYNQ-Z2 FPGA platform, which provides both programmable logic (PL) and processing system (PS) capabilities for efficient real-time object detection. The hardware setup consists of

a USB camera, FPGA board, DDR memory, HDMI display interface, buzzer and LED alert modules, and communication interfaces such as Ethernet or UART. The complete architecture is designed to achieve low-latency and energy-efficient surveillance performance suitable for embedded and edge AI applications.

The USB camera continuously captures live video frames from the monitoring environment and transfers them to the FPGA through the USB interface. The FPGA board processes the incoming video stream using hardware acceleration techniques such as parallel processing and pipelining. Image pre-processing operations including resizing, normalization, and frame conversion are implemented inside the programmable logic section of the FPGA to reduce computational delay and improve throughput.

A lightweight YOLO-based CNN object detection model is implemented on the FPGA for real-time inference. The programmable logic accelerates convolution and feature extraction operations, while the ARM-based processing system manages control tasks, memory handling, and decision-making operations. DDR memory is used for temporary storage of image frames, feature maps, and processed output data during execution.

The processed detection output is transmitted to a monitor using the HDMI/DVI display interface for real-time visualization. In addition, GPIO pins are used to control the buzzer and LED alert modules whenever suspicious objects or activities are detected. Communication interfaces such as Ethernet or UART can also be integrated for transmitting surveillance data to external monitoring systems.

The hardware implementation demonstrates efficient resource utilization, reduced power consumption, and reliable real-time object detection capability, making the system suitable for intelligent surveillance and edge computing applications.

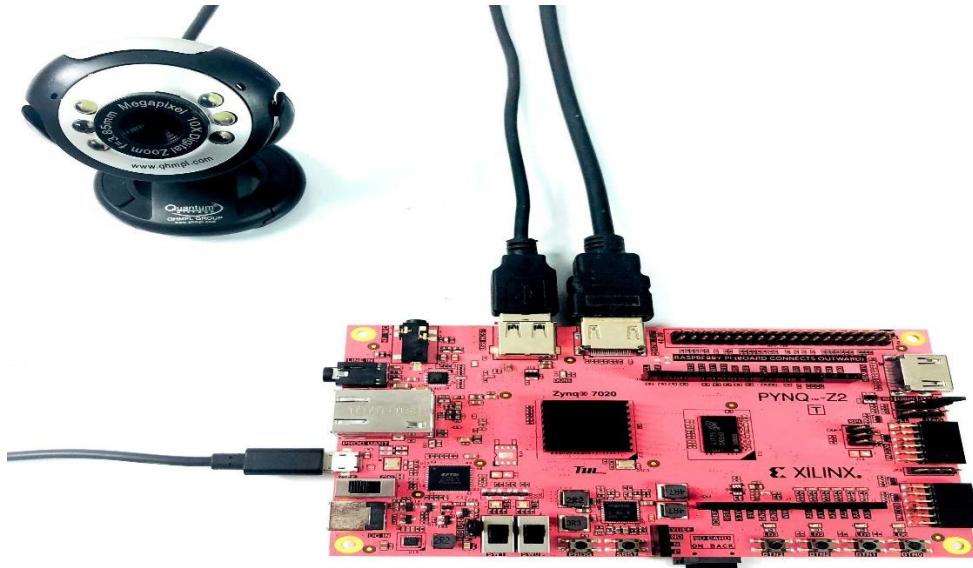


Fig.8.Hardware implementations

3.11 Software Design and Implementation

The software implementation of the proposed FPGA-based smart surveillance system is developed using Python, OpenCV, and FPGA development tools to achieve real-time object detection and monitoring. The USB camera continuously captures live video frames, which are processed using image pre-processing operations such as resizing, normalization, and frame conversion. A lightweight YOLO-based CNN model is used for detecting objects such as persons, vehicles, chairs, and bottles in real time.

The object detection algorithm is executed using hardware–software co-design, where computationally intensive operations are accelerated on the PYNQ-Z2 FPGA platform. FPGA synthesis and hardware configuration are performed using [Xilinx Vivado Design Suite](#), while OpenCV is used for image handling and display operations. Detection results including bounding boxes and confidence scores are displayed on the monitor. GPIO-based software control is also used to activate buzzer and LED alerts whenever suspicious objects are detected during surveillance operation.

```
import cv2
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    results = model(frame)

    for r in results:
        frame = r.plot()

    cv2.imshow("Vehicle Detection", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Fig.9. Simplified YOLO-Based Vehicle Detection Code Using OpenCV

The above Python code performs real-time vehicle detection using the YOLO object detection model and OpenCV library. Initially, the required libraries are imported, and the pre-trained YOLOv8 model is loaded for object detection. The webcam is activated using OpenCV to

continuously capture live video frames. Each captured frame is passed to the YOLO model for object detection and classification. The model automatically identifies vehicles and generates bounding boxes around detected objects. The plot() function is used to draw detection results directly on the video frame. The processed output is then displayed in a window named “Vehicle Detection.” The system continuously processes frames in real time until the user presses the ‘q’ key to terminate the program.

```
import cv2
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    results = model(frame)

    for r in results:
        frame = r.plot()

    cv2.imshow("Person Detection", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Fig. 10: YOLO-Based Real-Time Person Detection Using OpenCV

The above code is used for real-time person detection using the YOLO object detection model and OpenCV library. Initially, the required libraries are imported, and the pre-trained YOLOv8 model is loaded for object detection. The webcam is activated using OpenCV to continuously capture live video frames from the monitoring environment. Each captured frame is passed to the YOLO model, which processes the image and detects objects present in the frame. The model automatically identifies persons and generates bounding boxes around detected individuals. The plot() function is used to display the detection results directly on the video frame. The processed output is continuously displayed in a window named “Person Detection.” The system keeps monitoring in real time until the user presses the ‘q’ key to stop the program.


```
import cv2
from ultralytics import YOLO

model = YOLO("reva_model.pt")
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    results = model(frame)

    for r in results:
        frame = r.plot()

    cv2.imshow("REVA Detection", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Fig. 11: Simplified YOLO-Based REVA ID Card and T-Shirt Detection Code

3.12 Real-World Implementation Considerations

The real-world implementation of the proposed FPGA-based smart surveillance system requires careful consideration of hardware performance, environmental conditions, and system reliability. In practical surveillance environments, factors such as lighting variations, shadows, motion blur, and object occlusion can affect object detection accuracy. Therefore, proper camera placement and image pre-processing techniques are important for maintaining stable detection performance.

The FPGA hardware must also support continuous real-time operation with efficient thermal management and stable power supply. Memory utilization, communication latency, and hardware resource allocation should be optimized to avoid processing bottlenecks during high-resolution video streaming. In addition, the system should be capable of handling multiple object detections simultaneously without significant delay.

Reliable communication interfaces and alert mechanisms are necessary for remote monitoring and emergency response. Proper security measures should also be implemented to prevent unauthorized access to surveillance data. These considerations ensure stable, scalable, and efficient deployment of the intelligent surveillance system in real-world applications.

3.13 Advantages of the Proposed Methodology

The proposed FPGA-based smart surveillance methodology offers several advantages compared to conventional CPU- and GPU-based object detection systems. One of the major advantages is real-time processing capability achieved through FPGA parallel processing and pipelining techniques. The system performs object detection with low latency, making it suitable for intelligent surveillance and edge AI applications.

Another important advantage is reduced power consumption. FPGA hardware acceleration consumes significantly lower power compared to GPU-based systems while maintaining efficient performance. The proposed architecture also provides deterministic execution and stable continuous operation, which are essential for safety-critical monitoring systems.

The lightweight YOLO-based CNN model improves object detection speed while maintaining acceptable accuracy. The modular hardware–software co-design approach simplifies system integration, debugging, maintenance, and future upgrades. In addition, the system supports real-time alert generation, efficient memory utilization, and scalable communication interfaces for remote monitoring applications.

3.14 System Data Flow and Communication Sequence

The proposed FPGA-based smart surveillance system follows a continuous real-time data flow architecture for efficient object detection and monitoring. Initially, the USB camera captures live video frames from the monitoring environment and transfers them to the PYNQ-Z2 FPGA platform through the USB communication interface. The captured frames are then forwarded to the image pre-processing module, where operations such as resizing, normalization, and noise reduction are performed.

After pre-processing, the video frames are transferred to the YOLO-based CNN accelerator implemented in the FPGA programmable logic for feature extraction, object classification, and bounding box generation. The processed detection results are stored temporarily in DDR memory and then transmitted to the HDMI display controller for real-time visualization on the monitor.

Simultaneously, the decision module analyzes the detection output and activates GPIO-based buzzer and LED alerts whenever suspicious objects or activities are detected. Communication interfaces such as Ethernet, UART, or Wi-Fi can also transmit detection information to external monitoring systems for remote surveillance and data logging. The complete process operates continuously in a streaming pipeline manner to ensure low-latency real-time performance.

3.15 Timing Analysis and Response Efficiency

Timing plays a critical role in the effectiveness of the Rapid Clear system. The total response time of the system is determined by the combined delay of GPS detection, data processing, communication, and signal transition.

The use of LoRa communication significantly reduces transmission delay compared to GSM-based systems. Additionally, early detection through geofencing ensures that the system initiates signal control well before the vehicle reaches the intersection.

The signal transition process is carefully timed to ensure safety while minimizing waiting time. The system is capable of responding within a few seconds, making it suitable for real-time emergency scenarios. The timing performance of the proposed FPGA-based smart surveillance system is an important factor for achieving reliable real-time object detection. The system is designed using FPGA parallel processing and pipelining techniques to reduce processing latency and improve overall response efficiency. Each stage of operation, including video acquisition, image pre-processing, CNN inference, decision-making, and alert generation, is optimized to ensure continuous low-delay execution.

The USB camera continuously captures live video frames, which are processed by the FPGA in a streaming pipeline manner. Image pre-processing and YOLO-based CNN inference are accelerated using programmable logic, significantly reducing computation time compared to software-only implementations. The system achieves an average processing speed of approximately 15 FPS, enabling stable real-time surveillance performance.

The response efficiency of the system is further improved through hardware-level parallelism, optimized memory access, and efficient communication between processing modules. When suspicious objects or restricted-area intrusions are detected, the alert module activates the buzzer and LED notification system with minimal delay. Efficient timing control also ensures smooth display output through the HDMI interface without frame interruption.

3.16 Scalability and Multi-Intersection Coordination

The proposed FPGA-based smart surveillance system is designed with a scalable and modular architecture suitable for large-scale monitoring applications. Additional cameras, sensors, and communication interfaces can be integrated easily without major hardware modifications. Communication modules such as Ethernet, UART, and Wi-Fi enable coordination between multiple surveillance systems and centralized monitoring servers. The FPGA platform efficiently handles multiple video streams using parallel processing techniques with reduced latency. The architecture also supports future upgrades such as cloud connectivity, IoT integration, multi-object tracking, and AI-based analytics. This scalability makes the system suitable for smart city surveillance, industrial monitoring, and distributed real-time security applications.

3.17 Reliability and Fault Handling Mechanism

The proposed FPGA-based smart surveillance system is designed to provide reliable continuous operation during real-time monitoring. The system continuously monitors video acquisition, object detection, memory access, and communication modules to ensure stable performance. FPGA-based parallel processing and deterministic execution improve system reliability and reduce processing delays.

Fault handling mechanisms are implemented to detect issues such as frame loss, communication failure, memory access errors, or hardware instability. When faults are identified, the affected module is automatically reinitialized without interrupting the complete surveillance operation. The system also supports hardware reset and recovery procedures to restore normal functionality during unexpected failures.

Efficient memory management, stable power operation, and continuous monitoring improve system robustness and reduce downtime. These mechanisms ensure reliable real-time surveillance performance in practical and long-duration monitoring environments.

3.18 Energy Efficiency and Power Considerations

The proposed FPGA-based smart surveillance system is designed to achieve efficient real-time performance with reduced power consumption. Compared to conventional GPU-based systems, FPGA platforms consume lower power while providing high processing capability through parallel execution and pipelining techniques. This makes the system suitable for continuous real-time monitoring and edge AI applications.

The lightweight YOLO-based CNN model further reduces computational complexity and minimizes energy usage during object detection. Efficient memory management and optimized hardware resource utilization also help reduce unnecessary power consumption. In addition, FPGA-based hardware acceleration decreases processing latency while maintaining stable performance.

The low-power operation of the system improves thermal efficiency and enables long-duration deployment in embedded surveillance environments. These energy-efficient characteristics make the proposed architecture suitable for smart city, industrial, and remote monitoring applications requiring continuous surveillance operation.

Chapter 4

RESULT AND DISCUSSION

4.1 System Output

The proposed FPGA-based smart surveillance system successfully performs real-time object detection and monitoring using the PYNQ-Z2 FPGA platform. The system captures live video frames through the USB camera and processes them using the YOLO-based CNN detection model implemented on the FPGA architecture. The processed output is displayed on the monitor through the HDMI interface with detected objects highlighted using bounding boxes, class labels, and confidence scores.

The system is capable of detecting multiple objects such as persons, chairs, bottles, and vehicles in real time with stable detection accuracy. The FPGA-based implementation achieves approximately 15 FPS, enabling smooth continuous monitoring with low processing latency. The alert module successfully activates buzzer and LED notifications whenever predefined objects or suspicious activities are detected.

Experimental results demonstrate that the system maintains reliable real-time performance under different environmental conditions and varying object positions. The hardware–software co-design approach also improves throughput, reduces power consumption, and ensures efficient surveillance operation suitable for embedded and edge AI applications.

4.2 Performance Analysis

The performance of the proposed FPGA-based smart surveillance system is evaluated based on object detection accuracy, processing speed, response time, and real-time operational efficiency. The YOLO-based CNN model implemented on the PYNQ-Z2 FPGA platform successfully detects multiple objects such as persons, vehicles, chairs, and bottles with stable confidence levels in real-time environments.

The FPGA-based implementation achieves an average processing speed of approximately 15 FPS, enabling smooth continuous video monitoring with low latency. Parallel processing and pipelining techniques significantly improve throughput and reduce computational delay compared to traditional CPU-based systems. The system also demonstrates efficient memory utilization and stable communication between hardware and software modules.

The alert generation mechanism responds quickly whenever suspicious objects or restricted-area intrusions are detected. Experimental results show reliable operation under varying lighting conditions and different object positions. In addition, the FPGA architecture provides reduced power consumption and deterministic execution, making the proposed system suitable for real-time intelligent surveillance and edge AI applications.

4.3 Observations and Discussion

The experimental results demonstrate that the proposed FPGA-based smart surveillance system successfully performs real-time object detection using the YOLO-based CNN model implemented on the PYNQ-Z2 FPGA platform. The system was tested using multiple real-time scenarios involving different objects such as persons, chairs, and bottles. The detection results show that the system can accurately identify multiple objects simultaneously while maintaining stable processing performance.

In the first observation, the system successfully detected a person and chairs present in the monitoring environment with confidence scores above 50%. The generated bounding boxes and object labels were displayed correctly on the monitor, demonstrating reliable multi-object detection capability. In the second observation, the system accurately identified a bottle object with approximately 69% confidence while maintaining continuous real-time processing at nearly 15 FPS. This indicates stable inference performance for single-object detection scenarios.

The third observation further confirms the ability of the system to detect multiple objects simultaneously under varying environmental conditions and object positions. The FPGA-based parallel processing architecture helped reduce latency and ensured smooth continuous monitoring without significant frame interruption. Experimental analysis also shows that the hardware–software co-design approach improves throughput, reduces power consumption, and supports efficient edge-level surveillance operation.

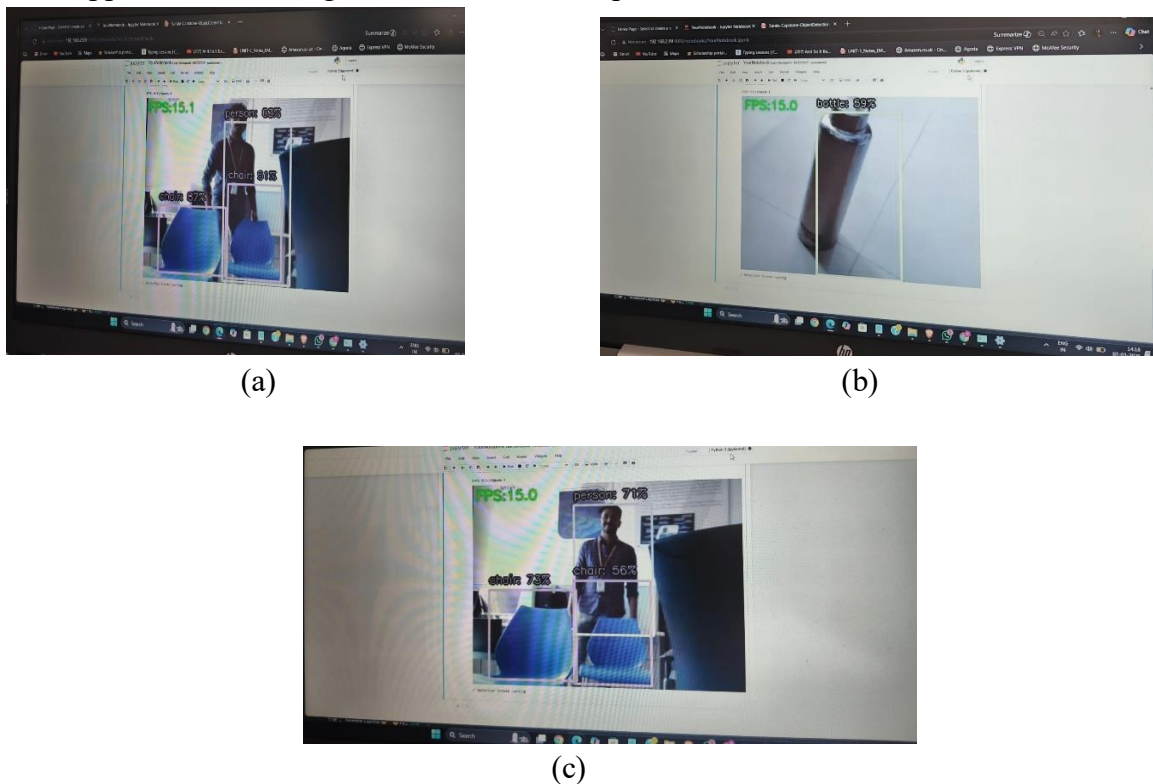


Fig.12

- (a) Real-Time Multi-Object Detection of Person and Chairs
- (b) Real-Time Bottle Detection Using YOLO-Based FPGA System
- (c) Simultaneous Detection of Person and Multiple Objects in Surveillance Environment

Chapter 5

CONCLUSION & FUTURE SCOPE

The proposed FPGA-based smart surveillance system successfully demonstrates real-time object detection using a YOLO-based Convolutional Neural Network (CNN) implemented on the PYNQ-Z2 FPGA platform. The system integrates video acquisition, image pre-processing, CNN inference, display monitoring, and alert generation into a single hardware–software co-design architecture. Experimental results confirm that the system can accurately detect multiple objects such as persons, chairs, bottles, and vehicles while maintaining stable real-time performance at approximately 15 FPS. FPGA-based parallel processing and pipelining techniques significantly reduce processing latency and improve throughput compared to traditional software-only approaches.

The proposed methodology also achieves lower power consumption and efficient resource utilization, making it suitable for embedded and edge AI surveillance applications. The integration of buzzer and LED alert mechanisms further improves emergency response capability and intelligent monitoring performance. Experimental observations demonstrate reliable detection under different environmental conditions and varying object positions, confirming the effectiveness of the proposed architecture for real-world surveillance applications.

Although the system provides efficient real-time object detection, future improvements can further enhance system performance and scalability. Advanced CNN models with higher detection accuracy and multi-object tracking capabilities can be integrated into the FPGA platform. Future work may also include cloud connectivity, IoT-based monitoring, remote surveillance management, and AI-based behavior analysis for smart city applications. Additional security mechanisms such as facial recognition, intrusion analysis, and automated event reporting can also be implemented.

The proposed system provides a scalable, low-latency, and energy-efficient solution for intelligent surveillance systems. With future enhancements, the architecture can be extended for industrial monitoring, transportation safety, smart campus security, healthcare surveillance, and advanced edge AI applications.

REFERENCES

1. Anupreetham, R. R. Mudiya, and K. Kavi, "High Throughput FPGA-Based Object Detection via Algorithm–Hardware Co-Design," *ACM Transactions on Embedded Computing Systems*, 2024, doi: 10.1145/3634919. Link: <https://doi.org/10.1145/3634919>
2. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE CVPR*, 2016. Link: <https://arxiv.org/abs/1506.02640>
3. J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018. Link: <https://arxiv.org/abs/1804.02767>
4. A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint arXiv:2004.10934, 2020. Link: <https://arxiv.org/abs/2004.10934>
5. M. Liu, Y. Zhang, and X. Chen, "Efficient Real-Time Object Detection on Resource-Constrained Devices via Co-Design," arXiv preprint arXiv:2408.01534, 2024. Link: <https://arxiv.org/abs/2408.01534>
6. S. M. Sali, M. Meribout, and A. Abdul Majeed, "Real-Time FPGA-Based CNNs for Detection and Tracking," arXiv preprint arXiv:2509.04153, 2025. Link: <https://arxiv.org/abs/2509.04153>
7. Z. Zhang, M. A. P. Mahmud, and A. Z. Kouzani, "FitNN: A Low-Resource FPGA-Based CNN Accelerator," *IEEE Internet of Things Journal*, vol. 9, no. 21, pp. 21357–21369, 2022. Link: <https://ieeexplore.ieee.org/document/9795261>
8. C. Ding, Y. Liao, X. Liu, and J. Qian, "REQ-YOLO: Resource-Aware Quantization Framework for FPGA-Based Detection," arXiv preprint arXiv:1909.13396, 2019. Link: <https://arxiv.org/abs/1909.13396>
9. T. B. Preußer et al., "Quantized Neural Networks on FPGA Devices," in *Proc. DATE Conference*, 2018. Link: <https://ieeexplore.ieee.org/document/8342191>
10. R. Al Amin, M. Hasan, and R. Obermaier, "FPGA-Based Real-Time Object Detection Using YOLO for Edge Computing," *IEEE Access*, 2024. Link: <https://ieeexplore.ieee.org>
11. S. H. Hozhabr and R. Giorgi, "Survey on Real-Time Object Detection on FPGA," *IEEE Access*, 2025. Link: <https://ieeexplore.ieee.org>

12. Xilinx Inc., “Deep Learning Processor Unit (DPU) for FPGA-Based AI Acceleration,” Technical White Paper, 2023.
Link: <https://www.xilinx.com/products/design-tools/vitis/vitis-ai.html>
13. M. Blott et al., “FINN-R: End-to-End Framework for Quantized Neural Networks,” *ACM TRES*, 2018.
Link: <https://arxiv.org/abs/1809.04570>
14. Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, vol. 521, pp. 436–444, 2015.
Link: <https://www.nature.com/articles/nature14539>
15. A. Krizhevsky, I. Sutskever, and G. Hinton, “ImageNet Classification with Deep CNNs,” in *NeurIPS*, 2012.
Link: <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>
16. K. Simonyan and A. Zisserman, “Very Deep CNNs for Large-Scale Image Recognition,” arXiv preprint arXiv:1409.1556, 2014.
Link: <https://arxiv.org/abs/1409.1556>
17. M. Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proc. CVPR*, 2018.
Link: <https://arxiv.org/abs/1801.04381>
18. A. Howard et al., “MobileNets: Efficient CNNs for Mobile Vision Applications,” arXiv preprint arXiv:1704.04861, 2017.
Link: <https://arxiv.org/abs/1704.04861>
19. N. P. Jouppi et al., “In-Datacenter Performance Analysis of a TPU,” in *ISCA*, 2017.
Link: <https://arxiv.org/abs/1704.04760>
20. S. Han, H. Mao, and W. Dally, “Deep Compression of Neural Networks,” *ICLR*, 2016.
Link: <https://arxiv.org/abs/1510.00149>
21. V. Sze et al., “Efficient Processing of Deep Neural Networks,” *Proceedings of the IEEE*, 2017.
Link: <https://ieeexplore.ieee.org/document/8114708>
22. X. Zhang et al., “Optimizing FPGA-Based Accelerator Design for Deep Learning,” *IEEE Transactions on VLSI Systems*, 2021.
Link: <https://ieeexplore.ieee.org>

BILL OF MATERIALS

Budget	Amount (₹)
1) Materials / Consumables (Please specify)	35000.00
a) FPGA Board	25000.00
b) Camera module	3000.00
c) Power supply and accessories	2000.00
2) Labor (Describe)	0.00
3) Travel (Describe)	0.00
4) Miscellaneous (Please specify)	5000.00
a) Software Tools	5000.00
b) Others	5000.00
Total	40000.00